15

BÖLÜM: K8S İŞ YÜKLERI & DAYANIKLILIK | MODULE: K8S WORKLOADS & SELF-HEALING RESILIENCE

Spring Boot Sağlık Kontrolleri: Liveness, Readiness ve Startup Probes

Spring Boot Health Probes: Liveness, Readiness, and Startup

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Doktor muayenesi zamanı! K8s üç farklı muayene yapar: 1) Startup Probe: 'Bebek doğdu mu, nefes aldı mı? (Spring Boot JVM ısınana kadar sabret!)' 2) Liveness Probe: 'Hasta yaşıyor mu? Kalbi durduysa (kilitlendiyse) elektroşok ver ve pod'u yeniden başlat!' 3) Readiness Probe: 'Müşteri kabul edebilir mi? Veritabanı bağlantısı koptuysa, masaya müşteri göndermeyi geçici olarak durdur!'

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Time for the medical checkup! Kubernetes acts as a vigilant doctor with 3 stethoscopes: 1) Startup Probe: 'Has the baby woken up yet? (Hold your horses while the heavy Java JVM warms up!)' 2) Liveness Probe: 'Is the heart deadlocked? If flatlining, kill and reboot it!' 3) Readiness Probe: 'Can the kitchen take orders? If the database connection drops, temporarily stop routing diners here!'

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Spring Boot Actuator 2.3+ ile `/actuator/health/liveness` ve `/actuator/health/readiness` uç noktaları yerleşik olarak gelir. 1) Startup Probe: Ağır uygulamaların başlaması uzun sürerse liveness'ın pod'u erken öldürmesini engeller. 2) Liveness Probe: Thread deadlock veya bellek tıkanıklığında Kubelet pod'u yeniden başlatır (`restartCount++`). 3) Readiness Probe: Başarısız olursa pod ÖLDÜRÜLMEZ; sadece K8s Service Endpoints listesinden çıkarılır, böylece kullanıcılara 502/503 hatası gitmez.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Spring Boot Actuator natively exposes `/actuator/health/liveness` and `/actuator/health/readiness`. 1) Startup Probe protects slow-booting JVM apps from premature liveness kills. 2) Liveness probe failures instruct Kubelet to terminate and restart the container. 3) Readiness probe failures do NOT kill the container; instead, kube-proxy strips the Pod IP from the Service endpoint slice, shielding users from HTTP 500 errors.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# application.yml:
management:
  endpoint:
    health:
      probes:
        enabled: true

# deployment.yaml Probes Bloğu:
startupProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  failureThreshold: 30
  periodSeconds: 2 # 60 saniye boyunca açılışı bekle
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  periodSeconds: 5
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Liveness Probe içinde veritabanı veya dış servis kontrolü yapmayın! Veritabanınız çökerse, kümedeki 100 Spring Boot pod'unun tamamı aynı anda liveness hatası verip yeniden başlar ve tüm küme kilitlenir (Cascading Failure). Veritabanı kontrolü sadece Readiness probe'da olmalıdır!

🇬🇧 WATCH OUT

NEVER check external dependencies (database, Redis) in your Liveness Probe! If the database stumbles, all 100 app pods will simultaneously reboot in a catastrophic cascading failure.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Startup Probe JVM uygulamalarının açılışındaki erken ölümleri önler.
- Liveness kilitlenen uygulamayı yeniden başlatır (Restart).
- Readiness hazır olmayan pod'a trafiğin akmasını engeller (Zero 500 Errors).

- Startup probes give heavy JVM applications generous breathing room to initialize.
- Liveness restarts frozen or deadlocked application processes.
- Readiness isolates unready pods from live customer traffic safely.